Support Ticket Similarity Assistant

Solution Design · FDE take-home case study · Solari Analytics

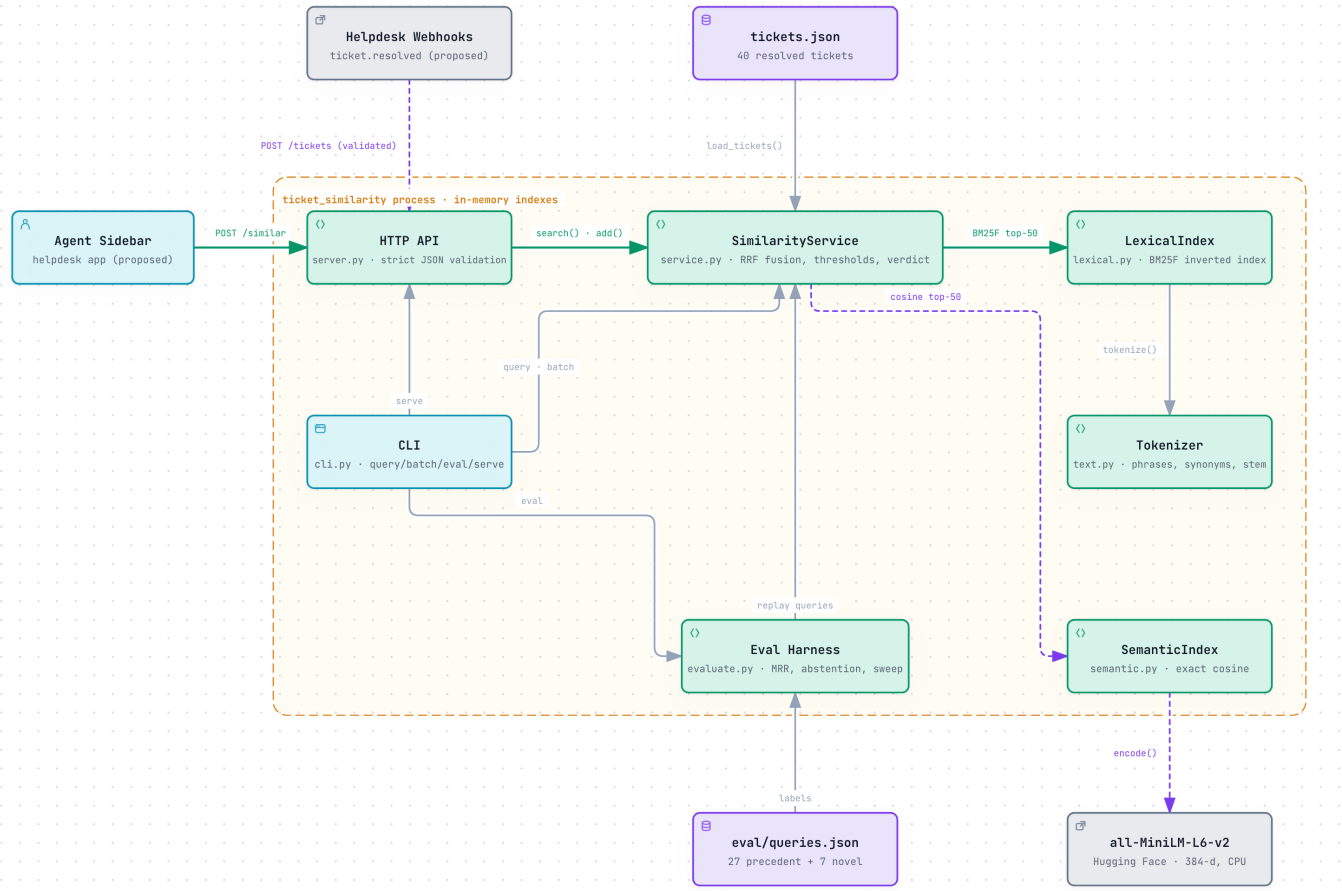
1. The problem, restated

Solari's support team takes ~150 tickets/day. Agents re-solve problems that were already solved and, worse, give customers inconsistent answers. When a ticket arrives, the agent should see the **few resolved tickets describing the same underlying problem, with how they were fixed**, and should be told plainly when **there is no precedent**, so a novel issue isn't answered with a confidently wrong past fix. Volume is expected to grow substantially, so the approach has to have a credible path past "fits in a laptop's memory."

2. Assumptions

Assumption	Why	If wrong
"Similar" = same symptom / root cause, so the fix transfers. Category alone is not similarity.	That's what saves agent time and keeps answers consistent.	Add category/product filters as facets; cheap.
Only `resolved` tickets with resolution notes are searchable.	An unresolved precedent gives the agent nothing to reuse.	Index open tickets separately as "related, in progress" (also useful for incident detection).
Volume: 150/day ≈ 55k/year. "A lot more" = ~3× within a year → design for **~200k** indexed tickets, with a named migration path beyond.	Concrete numbers make the scaling decision testable.	Above ~200k swap the in-memory index for Postgres FTS/OpenSearch + pgvector (§5).
Latency budget: **p95 < 300 ms** service time; suggestions render in the agent's sidebar when the ticket opens.	Interactive, but not keystroke-level.	Precompute on `ticket.created` webhook and cache; the budget becomes irrelevant.
English, short free text, no attachments. Agents are the consumers (human in the loop).	Matches the seed data.	Language detection + multilingual embedding model.
Precision over recall for anything labelled a match; abstain rather than mislead.	The customer's stated pain is inconsistent answers.	Thresholds are config, not code.
No relevance labels exist; I built a 34-query eval set myself (§4).	Need *some* ground truth to set thresholds instead of guessing.	Replace with agent click-feedback (§6).

3. Approach



Solid green: the main request path. Dashed purple: optional hybrid mode and the proposed helpdesk webhook. Orange boundary: one process with in-memory indexes. Interactive version: `architecture.html`.

Normalisation (`text.py`): lowercasing, contraction handling, phrase canonicalisation (`log in`/`sign-in` → `login`, `MFA`/`two-factor` → `2fa`, `times out` → `timeout`, `read-only` → `viewer`), ~15 domain synonyms, support-boilerplate stopwords (`customer`, `reports`, `noticed`…), a light suffix stemmer. 3-digit numbers are kept (`429`, `500`); other numbers are dropped.

Lexical retrieval (`lexical.py`): BM25F with per-field weights — subject 3.0, tags 2.0, description 1.0, resolution notes 0.4 — and per-field length normalisation, scored term-at-a-time off an inverted index. Resolution notes are deliberately down-weighted: they contain *cause* vocabulary ("SAML assertion", "idempotency keys") that a fresh ticket almost never uses, so a high weight mostly rewards accidental overlap.

Confidence / "no good match": BM25 scores are unbounded and not comparable across queries, so they can't be thresholded. Instead, confidence is **IDF-weighted query coverage**: the share of the query's information (sum of IDF) that the candidate explains. Terms never seen in the corpus get maximum IDF, so a ticket full of new vocabulary ("Snowflake", "warehouse") cannot reach high coverage and the service abstains. It still returns `nearest` for transparency, but never presents it as a match.

Semantic layer (`semantic.py`, `--mode hybrid`): `all-MiniLM-L6-v2` (384-d, CPU) over subject + description + tags; exact cosine via a single matvec. Lexical and dense rankings are merged with **Reciprocal Rank Fusion** (k=60), which is rank-based and needs no score calibration. Hybrid confidence = mean(coverage, cosine).

Verdict tiers — `strong_match` / `weak_match` / `no_match` — use thresholds calibrated per mode with a balanced-accuracy sweep (`eval --sweep`). Every match carries the resolution notes, matched terms and per-signal scores, so an agent (or I, in the demo) can see *why* it matched.

4. Evidence

Eval set (`eval/queries.json`): 27 queries with a known precedent (the 3 matchable example tickets + 24 paraphrases with deliberately shifted vocabulary) and 7 novel issues (NT-4 Snowflake, BigQuery connector, SOC 2 request, Android crash, …).

	lexical	hybrid
hit@1 / hit@3 / MRR	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00
novel correctly abstained	7/7	7/7
precedent wrongly abstained	1/27	0/27
separation margin (min precedent conf − max novel conf)	**0.02**	**0.11**
p50 latency (40 tickets)	0.06 ms	~4.7 ms

The ranking numbers are optimistic: I wrote the paraphrases after reading the data, and the set is tiny. The meaningful finding is the **margin**. Lexical ranks well, but its no-precedent decision sits on a razor's edge; embeddings are what make abstention reliable.

Scale (`scripts/benchmark.py`; synthetic corpus recombined from the 40 seeds, so postings are unrealistically dense — a worst case):

tickets	lexical p50 / p95	index build	memory
10k	7 / 11 ms	2 s	84 MB
55k (≈1 yr)	52 / 74 ms	13 s	459 MB
165k (≈3× growth)	169 / 232 ms	39 s	1.4 GB

Dense: 165k × 384 float32 = 253 MB; matvec + top-k 4.4 ms; query encoding 3.8 ms on CPU; indexing ~0.4 s per 1k tickets.

5. Trade-offs and why

- **Keyword vs embeddings vs LLM.** Lexical: zero dependencies, explainable, exact on tokens that matter in support (`429`, `Okta`, `SAML`), but blind to paraphrase ("terminated employee" vs "former employee"). Embeddings: robust to paraphrase and better calibrated, at the cost of a ~90 MB model, a torch dependency, less explainability, and re-embedding whenever the model changes. An LLM in the retrieval path costs 1–3 s, money per ticket and a data-governance review, for little gain at this size. **Decision:** lexical is the default (runs anywhere with stock Python); hybrid is one flag away, and is my production recommendation because abstention quality is what protects answer consistency.
- **Exact vs ANN vector search.** Exact search stays under ~25 ms up to ~1M vectors. ANN (HNSW) adds recall loss and operational surface for no benefit at this scale.
- **In-memory vs a search engine.** In-memory with rebuild-from-source at startup is simplest. The benchmark shows where it stops being fine: ~165k tickets (p95 ~230 ms, 1.4 GB of Python dict overhead, 40 s cold start). Past that, BM25 moves to Postgres FTS or OpenSearch and vectors to pgvector. `SimilarityService` is the seam, so callers don't change.
- **Abstain vs always show the top 3.** Showing a wrong precedent is worse than showing none, given the problem statement. The cost is occasionally hiding a useful but loosely-worded match (P-04 in lexical mode).
- **Hand-maintained synonyms.** Cheap and auditable, but easy to overfit. I kept the list small and generic; embeddings cover the long tail.
- **Strict input validation at the API and service boundary.** Malformed requests get a 400 instead of crashing, and each ticket is fully validated before any index is written. Before this, one malformed ticket could leave the ticket list and keyword index one entry apart, silently attributing later tickets to the wrong fix. Integer ticket IDs (as Zendesk uses) are accepted and normalised to strings rather than rejected.

- **Starter** `ticket_utils.py` **— reviewed, not used.** `keyword_overlap_score` divides by the query size only, so any long ticket containing the query's words scores 1.0. It has no stopwords (so "the"/"is" count as evidence), no term weighting, and no tags or resolution text. `find_top_matches` always returns `top_n`, even for nonsense queries, so it has no "no match" path at all.

6. With more time, data or budget

1. **Feedback loop first:** log impressions and an agent "this helped / not relevant" click. That yields real labels to re-tune weights and thresholds, and lets us measure the business metrics: handle time and answer consistency, A/B against a control queue.
2. **Integration:** Zendesk/Freshdesk sidebar app driven by `ticket.created` / `ticket.solved` webhooks; auth; PII scrubbing before anything is embedded.
3. **Freshness:** recency decay, and a "fix shipped" flag so workarounds for since-fixed bugs stop being recommended. Collapse near-duplicate precedents into one canonical known-issue entry.
4. **LLM layer, gated on** `strong_match`**:** a one-line suggested fix synthesised from the top matches, with citations; a cross-encoder reranker over the top 20.
5. **Novel-issue signal:** a burst of `no_match` tickets sharing new vocabulary (five Snowflake tickets in a day) should page someone as a likely incident.
6. **Ops:** persisted index, multi-tenant isolation, and the eval set running in CI as a regression gate.